

# Downloading the dataset

```
In [4]: # Download dataset
!gdown --id 1CuAtT7t1RWq-ep9RGPZAX3zMtFMzdZXV -O PlantVillage.zip

# Unzip the downloaded file
!unzip -q PlantVillage.zip
```

/usr/local/lib/python3.12/dist-packages/gdown/\_\_main\_\_.py:140: FutureWarning: Option `--id` was deprecated in version 4.3.1 and will be removed in 5.0. You don't need to pass it anymore to use a file ID.

warnings.warn(

Downloading...

From (original): <https://drive.google.com/uc?id=1CuAtT7t1RWq-ep9RGPZAX3zMtFMzdZXV>

From (redirected): <https://drive.google.com/uc?id=1CuAtT7t1RWq-ep9RGPZAX3zMtFMzdZXV&confirm=t&uuid=b3611644-7221-4a79-846e-8a73c90a960b>

To: /content/PlantVillage.zip

100% 690M/690M [00:09<00:00, 73.3MB/s]

# Importing libraries

```
In [ ]: import nbconvert
import numpy as np
import pickle
import cv2
from os import listdir
from sklearn.preprocessing import LabelBinarizer
from keras.models import Sequential
from keras.layers import BatchNormalization
from keras.layers import Conv2D
from keras.layers import MaxPooling2D
from keras.layers import Activation, Flatten, Dropout, Dense
from keras import backend as K
from keras.optimizers import Adam
from keras.preprocessing import image
from keras.preprocessing.image import img_to_array
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

import tensorflow as tf
```

```
In [12]: EPOCHS = 25
INIT_LR = 1e-3
BS = 32
default_image_size = tuple((256, 256))
image_size = 0
directory_root = 'plantvillage/'
width=256
```

```
height=256
depth=3
```

## Data preprocessing

```
In [13]: def convert_image_to_array(image_dir):
    try:
        image = cv2.imread(image_dir)
        if image is not None :
            image = cv2.resize(image, default_image_size)
            return img_to_array(image)
        else :
            return np.array([])
    except Exception as e:
        print(f"Error : {e}")
        return None
```

- Fetch images

```
In [14]: image_list, label_list = [], []
    try:
        print("[INFO] Loading images ...")
        root_dir = listdir(directory_root)
        for directory in root_dir :
            # remove .DS_Store from list
            if directory == ".DS_Store" :
                root_dir.remove(directory)

        for plant_folder in root_dir :
            plant_disease_folder_list = listdir(f"{directory_root}/{plant_folder}")

            for disease_folder in plant_disease_folder_list :
                # remove .DS_Store from list
                if disease_folder == ".DS_Store" :
                    plant_disease_folder_list.remove(disease_folder)

            for plant_disease_folder in plant_disease_folder_list:
                print(f"[INFO] Processing {plant_disease_folder} ...")
                plant_disease_image_list = listdir(f"{directory_root}/{plant_folder}/{plant_disease_folder}")

                for single_plant_disease_image in plant_disease_image_list :
                    if single_plant_disease_image == ".DS_Store" :
                        plant_disease_image_list.remove(single_plant_disease_image)

                for image in plant_disease_image_list[:200]:
                    image_directory = f"{directory_root}/{plant_folder}/{plant_disease_folder}/{image}"
                    if image_directory.endswith(".jpg") == True or image_directory.endswith(".png") == True:
                        image_list.append(convert_image_to_array(image_directory))
                        label_list.append(plant_disease_folder)

        print("[INFO] Image loading completed")
    except Exception as e:
        print(f"Error : {e}")
```

```
[INFO] Loading images ...
[INFO] Processing Tomato_Early_blight ...
[INFO] Processing Potato__Late_blight ...
[INFO] Processing Tomato__Target_Spot ...
[INFO] Processing Tomato_Spider_mites_Two_spotted_spider_mite ...
[INFO] Processing Tomato_healthy ...
[INFO] Processing Potato__healthy ...
[INFO] Processing Potato__Early_blight ...
[INFO] Processing Tomato_Septoria_leaf_spot ...
[INFO] Processing Tomato__Tomato_mosaic_virus ...
[INFO] Processing Pepper__bell__healthy ...
[INFO] Processing Tomato_Leaf_Mold ...
[INFO] Processing Tomato_Bacterial_spot ...
[INFO] Processing Tomato__Tomato_YellowLeaf__Curl_Virus ...
[INFO] Processing Pepper__bell__Bacterial_spot ...
[INFO] Processing Tomato_Late_blight ...
[INFO] Image loading completed
```

```
In [16]: image_size = len(image_list)
         print(image_size)
```

2952

- Transform image labels

```
In [17]: label_binarizer = LabelBinarizer()
         image_labels = label_binarizer.fit_transform(label_list)
         pickle.dump(label_binarizer, open('label_transform.pkl', 'wb'))
         n_classes = len(label_binarizer.classes_)
```

```
In [18]: print(label_binarizer.classes_)
```

```
['Pepper__bell__Bacterial_spot' 'Pepper__bell__healthy'
 'Potato__Early_blight' 'Potato__Late_blight' 'Potato__healthy'
 'Tomato_Bacterial_spot' 'Tomato_Early_blight' 'Tomato_Late_blight'
 'Tomato_Leaf_Mold' 'Tomato_Septoria_leaf_spot'
 'Tomato_Spider_mites_Two_spotted_spider_mite' 'Tomato__Target_Spot'
 'Tomato__Tomato_YellowLeaf__Curl_Virus' 'Tomato__Tomato_mosaic_virus'
 'Tomato_healthy']
```

- Splitting data

```
In [19]: np_image_list = np.array(image_list, dtype=np.float16) / 225.0
```

```
In [20]: x_train, x_test, y_train, y_test = train_test_split(np_image_list, image_labels, te
```

## Creating model

```
In [21]: aug = tf.keras.preprocessing.image.ImageDataGenerator(
         rotation_range=25, width_shift_range=0.1,
         height_shift_range=0.1, shear_range=0.2,
```

```
zoom_range=0.2,horizontal_flip=True,  
fill_mode="nearest")
```

```
In [22]: model = Sequential()  
inputShape = (height, width, depth)  
chanDim = -1  
if K.image_data_format() == "channels_first":  
    inputShape = (depth, height, width)  
    chanDim = 1  
model.add(Conv2D(32, (3, 3), padding="same", input_shape=inputShape))  
model.add(Activation("relu"))  
model.add(BatchNormalization(axis=chanDim))  
model.add(MaxPooling2D(pool_size=(3, 3)))  
model.add(Dropout(0.25))  
model.add(Conv2D(64, (3, 3), padding="same"))  
model.add(Activation("relu"))  
model.add(BatchNormalization(axis=chanDim))  
model.add(Conv2D(64, (3, 3), padding="same"))  
model.add(Activation("relu"))  
model.add(BatchNormalization(axis=chanDim))  
model.add(MaxPooling2D(pool_size=(2, 2)))  
model.add(Dropout(0.25))  
model.add(Conv2D(128, (3, 3), padding="same"))  
model.add(Activation("relu"))  
model.add(BatchNormalization(axis=chanDim))  
model.add(Conv2D(128, (3, 3), padding="same"))  
model.add(Activation("relu"))  
model.add(BatchNormalization(axis=chanDim))  
model.add(MaxPooling2D(pool_size=(2, 2)))  
model.add(Dropout(0.25))  
model.add(Flatten())  
model.add(Dense(1024))  
model.add(Activation("relu"))  
model.add(BatchNormalization())  
model.add(Dropout(0.5))  
model.add(Dense(n_classes))  
model.add(Activation("softmax"))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:  
113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When  
using Sequential models, prefer using an `Input(shape)` object as the first layer in  
the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
In [23]: model.summary()
```

**Model: "sequential"**

| Layer (type)                               | Output Shape         | Param #    |
|--------------------------------------------|----------------------|------------|
| conv2d (Conv2D)                            | (None, 256, 256, 32) | 896        |
| activation (Activation)                    | (None, 256, 256, 32) | 0          |
| batch_normalization (BatchNormalization)   | (None, 256, 256, 32) | 128        |
| max_pooling2d (MaxPooling2D)               | (None, 85, 85, 32)   | 0          |
| dropout (Dropout)                          | (None, 85, 85, 32)   | 0          |
| conv2d_1 (Conv2D)                          | (None, 85, 85, 64)   | 18,496     |
| activation_1 (Activation)                  | (None, 85, 85, 64)   | 0          |
| batch_normalization_1 (BatchNormalization) | (None, 85, 85, 64)   | 256        |
| conv2d_2 (Conv2D)                          | (None, 85, 85, 64)   | 36,928     |
| activation_2 (Activation)                  | (None, 85, 85, 64)   | 0          |
| batch_normalization_2 (BatchNormalization) | (None, 85, 85, 64)   | 256        |
| max_pooling2d_1 (MaxPooling2D)             | (None, 42, 42, 64)   | 0          |
| dropout_1 (Dropout)                        | (None, 42, 42, 64)   | 0          |
| conv2d_3 (Conv2D)                          | (None, 42, 42, 128)  | 73,856     |
| activation_3 (Activation)                  | (None, 42, 42, 128)  | 0          |
| batch_normalization_3 (BatchNormalization) | (None, 42, 42, 128)  | 512        |
| conv2d_4 (Conv2D)                          | (None, 42, 42, 128)  | 147,584    |
| activation_4 (Activation)                  | (None, 42, 42, 128)  | 0          |
| batch_normalization_4 (BatchNormalization) | (None, 42, 42, 128)  | 512        |
| max_pooling2d_2 (MaxPooling2D)             | (None, 21, 21, 128)  | 0          |
| dropout_2 (Dropout)                        | (None, 21, 21, 128)  | 0          |
| flatten (Flatten)                          | (None, 56448)        | 0          |
| dense (Dense)                              | (None, 1024)         | 57,803,776 |
| activation_5 (Activation)                  | (None, 1024)         | 0          |

|                                               |              |        |
|-----------------------------------------------|--------------|--------|
| batch_normalization_5<br>(BatchNormalization) | (None, 1024) | 4,096  |
| dropout_3 (Dropout)                           | (None, 1024) | 0      |
| dense_1 (Dense)                               | (None, 15)   | 15,375 |
| activation_6 (Activation)                     | (None, 15)   | 0      |

**Total params:** 58,102,671 (221.64 MB)

**Trainable params:** 58,099,791 (221.63 MB)

**Non-trainable params:** 2,880 (11.25 KB)

## Training model

```
In [25]: opt = Adam(learning_rate=INIT_LR, decay=INIT_LR / EPOCHS)
# distribution
model.compile(loss="binary_crossentropy", optimizer=opt, metrics=["accuracy"])
```

/usr/local/lib/python3.12/dist-packages/keras/src/optimizers/base\_optimizer.py:86: UserWarning: Argument `decay` is no longer supported and will be ignored.  
warnings.warn(

```
In [27]: history = model.fit(
    aug.flow(x_train, y_train, batch_size=BS),
    validation_data=(x_test, y_test),
    steps_per_epoch=len(x_train) // BS,
    epochs=EPOCHS, verbose=1
)
```

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data\_adapters/py\_dataset\_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.  
self.\_warn\_if\_super\_not\_called()

Epoch 1/25

73/73 ————— 749s 10s/step - accuracy: 0.1654 - loss: 0.7959 - val\_accuracy: 0.0846 - val\_loss: 0.5086

Epoch 2/25

1/73 ————— 11:17 9s/step - accuracy: 0.0938 - loss: 0.4944

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/epoch\_iterator.py:116: UserWarning: Your input ran out of data; interrupting training. Make sure that your dataset or generator can generate at least `steps\_per\_epoch \* epochs` batches. You may need to use the `.repeat()` function when building your dataset.  
self.\_interrupted\_warning()



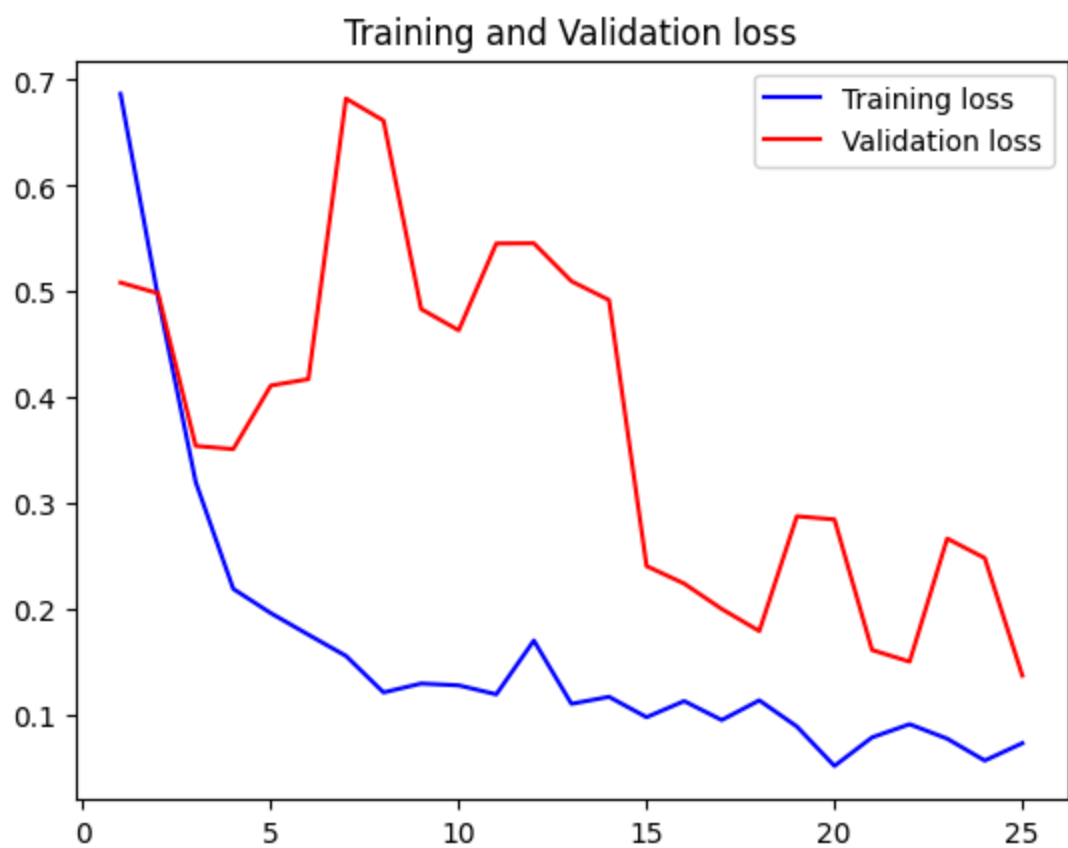
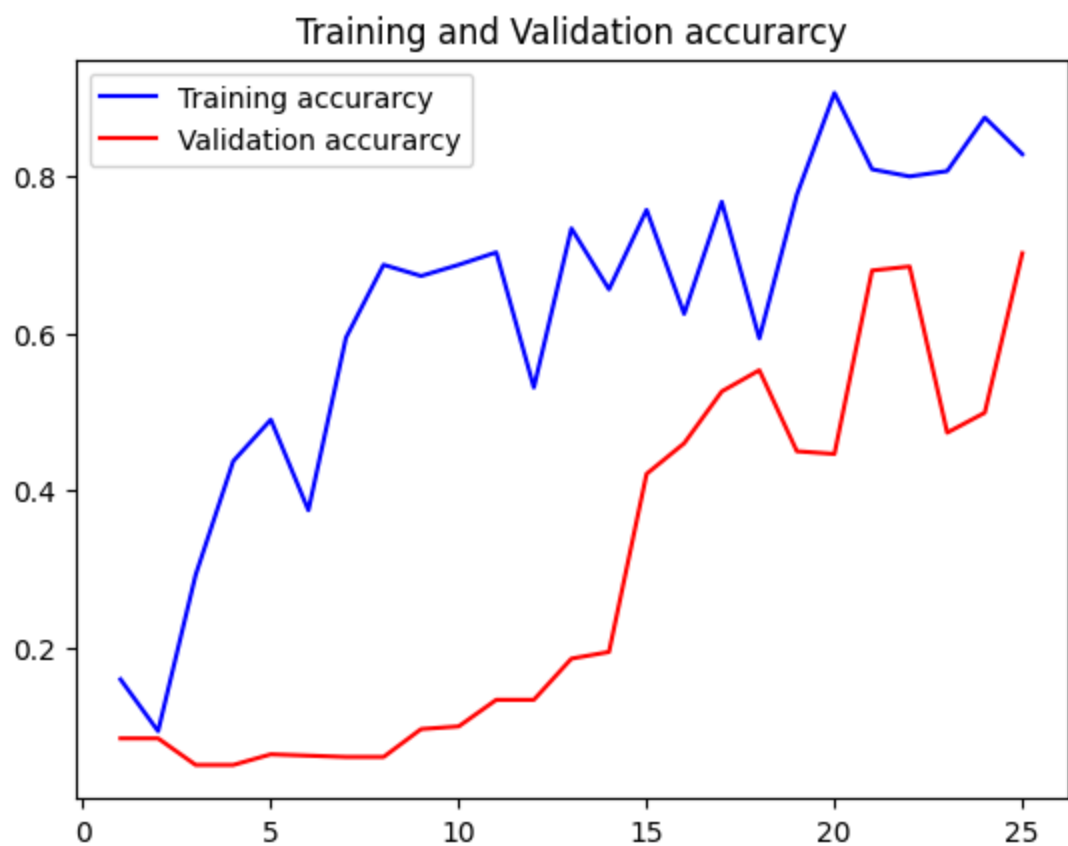
curacy: 0.4467 - val\_loss: 0.2849  
 Epoch 21/25  
 73/73 ————— 767s 10s/step - accuracy: 0.8115 - loss: 0.0796 - val\_acc  
 uracy: 0.6802 - val\_loss: 0.1616  
 Epoch 22/25  
 73/73 ————— 40s 451ms/step - accuracy: 0.8000 - loss: 0.0915 - val\_ac  
 curacy: 0.6853 - val\_loss: 0.1506  
 Epoch 23/25  
 73/73 ————— 756s 10s/step - accuracy: 0.8108 - loss: 0.0763 - val\_acc  
 uracy: 0.4738 - val\_loss: 0.2668  
 Epoch 24/25  
 73/73 ————— 50s 569ms/step - accuracy: 0.8750 - loss: 0.0572 - val\_ac  
 curacy: 0.4992 - val\_loss: 0.2484  
 Epoch 25/25  
 73/73 ————— 721s 10s/step - accuracy: 0.8343 - loss: 0.0723 - val\_acc  
 uracy: 0.7022 - val\_loss: 0.1374

```

In [30]: acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(1, len(acc) + 1)
#Train and validation accuracy
plt.plot(epochs, acc, 'b', label='Training accuracy')
plt.plot(epochs, val_acc, 'r', label='Validation accuracy')
plt.title('Training and Validation accuracy')
plt.legend()

plt.figure()
#Train and validation loss
plt.plot(epochs, loss, 'b', label='Training loss')
plt.plot(epochs, val_loss, 'r', label='Validation loss')
plt.title('Training and Validation loss')
plt.legend()
plt.show()
  
```





```
In [31]: scores = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {scores[1]*100}")
```

# Documentation

---

## Summary

In this project we are classifying plant diseases by analyzing images of leaves.

The approach chosen was to train a model to classify the images.

---

## Implementation

The first step is fetching the data and storing it in an easy to use format, such as an array.

Then comes the preprocessing, where each label is binarized and the array of images is converted to an np array, which is then split into train and test sets.

The model we are using is a sequential model which uses multiple RELU activations and batch normalizations, followed by a softmax activation at the end.

To train, we are using 25 epochs, each with 73 steps and an Adam optimizer.

---

## Results

During training, the model achieved up to 87% accuracy on the training data.

When using the test data it achieved a 70% accuracy.